

Software Architecture

Braxton Vogel - Prepared for Jeffrey Palermo

1.) Architectural Styles

- **Monolithic Architecture**

It's the entire application; UI, business logic, and data access. It's built and deployed as a single unit.

Strengths: Simple to develop, test, and deploy early on; no network overhead between internal components

Trade offs: can be hard to scale various parts independently; a fault in one module can take down the entire application/app. Maintenance also gets harder as the codebase grows.

- **Microservices Architecture**

This application is decomposed into small, independently deployable services, each owning both its own data and communicating over the networks (HTTP/gRPC or messaging)

Strengths: teams are able to work independently; each service both scales and deploys on its own; any failures are isolated

Trade offs: Operational complexity (service discovery, monitoring, distributed tracing); network latency can be tough to maneuver; cross service data consistency has a tendency of being hard

- **Service Oriented Architecture (SOA)**

A predecessor to microservices where as services communicate via a shared enterprise service bus (ESB), this allows more centralized infrastructure than microservices typically use

Strengths: Encourages reuse of services across the organization; centralized governance

Trade offs: The ESB can become a bottleneck or possibly a single point of failure; heavier than microservices

Personal Note: I find this one the most interesting due to being able to reuse services as an independent application that anyone can talk to (Sorta like a function in java/python, but running as its own separate service over the network)

- **Layered (N-tier) Architecture**

The code here is organized into sorta horizontal layers; presentation, business logic, and data access. Where as each layer only talks to the layer directly below it

Strengths: Has clear separation of concerns; familiar and easy for newer, less experienced developers to learn

Trade offs: Can lead to tight coupling between layers; a change has the ability to ripple across the stack

- **Event Driven Architecture (EDA)**

The components communicate via producing and consuming events, typically through a message broker (Kafka, RabbitMQ), rather than direct calls

Strengths: Loose coupling; well suited to real time and asynchronous workflows; scales well

Trade offs: Can be harder to trace and debug since there's no single call stack; eventual consistency complicates reasoning about state

- **Serverless / Function as a Service (FaaS)**

Code runs as an individual function triggered by events, with the cloud provider managing both infrastructure and scaling

Strengths: no server management; scales to zero; pay per use pricing

Trade offs: Cold start latency; vendor lock in; can be harder to both test and debug locally; and can get costly at steady, high volumes

Personal Note: I find this interesting because I never thought to being able to do something like this without a traditional server setup

- **Space Based Architecture**

Designed to avoid database bottlenecks by keeping data in memory across a distributed 'space' of processing units, used for high scale, and high throughput systems

Strengths: Excellent horizontal scalability under super heavy loads

Trade offs: Complex to both implement and reason about

2.) **Architectural Patterns (Application Internal)**

How a single application is structured internally

<u>Pattern</u>	<u>What it does</u>
MVC	Separates data (model), UI, & control logic (controller). Classic web frameworks like Rails, Django, ASP.NET.
MVVM	Model View ViewModel; common in UI frameworks with data binding. Such as WPF, Android, & Vue style reactive UIs
Hexagonal (Ports & Adapters)	Isolates core business logic from external systems (Database, UI, APIs) via interfaces, so infrastructure can easily be swapped
Clean / Onion Architecture	Concentric layers with dependencies pointing inward towards business rules, keeping both framework and databases as replaceable details
CQRS	Command query responsibility Segregation which separates read and write models, often paired with event sourcing for workloads with very different read/write scaling needs

3.) **Deployment & Infrastructure Patterns**

The lower level models that architectural styles from above are often built on

- **Client Server:** The foundational request/response model most of the styles above build on
- **Peer to Peer (P2P):** No central server; nodes act as both client & server (BitTorrent, blockchain networks) (Learned about this in my computer networking class)
- **Pipe and Filter:** Data flows through a sequence of processing takes. Most common in compilers and data pipelines

4.) Choosing an Architecture

While there is no actual universally “best” architecture the right choice depends on the variables; team size, expected scale, deployment constraints, and how much optional complexity the team can absorb. A few rules of thumb I’ve uncovered:

- Start monolithic for new products with small teams and uncertain requirements; it's faster to iterate on and easier to refactor later
- Move toward microservices or event-driven models once distinct parts of the system need to scale, deploy, or fail independently
- Use hexagonal/clean architecture principles inside any style to keep business logic testable and decoupled from frameworks
- Reach for serverless for spiky, event-driven workloads where paying for idle capacity doesn't make sense

5.) Where Architecture Runs: Embedded & Edge Devices

Architecture isn't only about how software is organized, it's also about what hardware it's built to run on.

Everything covered so far assumes software running on general-purpose servers or cloud infrastructure. A separate consideration is embedded and edge architecture, software built to run directly on small, low-power, purpose-built hardware rather than in a data center.

- **Ex: ARM Based Technology**

Based on the small camera device Jeffrey showed me on our first meeting; a small camera device using ARM technology, looking for a second real world example; I found Valve’s Steam frame VR headset. It runs on a Qualcomm Snapdragon 8 Gen 3 chip; an ARM processor with SteamOS; Valve’s Linux based operating system. This runs natively on that ARM hardware rather than on a traditional x86 PC chip

- **Standalone, on device computing:** The headset can run games entirely on its own hardware without needing to be connected to a PC, similar to how an edge device processes data locally instead of relying on a remote server
- **Resource Constraints:** Unlike a desktop GPU the headset’s ARM chip has to balance performance against battery life, heat, and weight; since it’s worn on someone’s head, not sitting in a case with a fan

- **Bridging Architectures:** Most PC games are built for an x86 chip, not ARM. So Valve built a translation layer (FEX) that converts x86 code to be able to run on the headset's ARM processor additionally a streaming mode

6.) Why Markups Are So Important Before Starting A Project

Before writing a single line of code on work handed down by an employer or client, the requirements, mockups, specs, or existing materials should be marked up first; annotated, questioned, and clarified in writing. Skipping this step and jumping straight into implementation is treated as a defect in the process itself, not just a risk.

- Jeffrey Palermo's *"The Five Pillars: Leadership For Effective Custom Software"* Book (Says when there isn't a markup it should be considered a defect)

- What Markups Are

A markup is a written pass over what was given to you; a spec, a set of designs, a ticket, an existing codebase, where you annotate it directly: flagging ambiguities, calling out missing information, noting assumptions you're making, and raising questions before any work begins. It becomes a shared reference between you and the person who assigned the work, rather than something that lives only in your head.

- Why Markups should be treated as mandatory, not optional

- **Catches vagueness early:** Requirements are rarely complete or explicit. A markup surfaces gaps while they're easy to fix; before coding, design, or architecture has been built on top of a wrong assumption
- **Creates a history:** If a question wasn't ever asked that means there's no record that an assumption was made. A markup documents exactly what was understood at the outset, this protects both the employer and person doing the work
- **Prevents Costly Rework:** Uncovering a misunderstanding after the work is done is far more expensive than discovering it before starting; in time, trust, and morale
- **Shows Professionalism:** Marking something up shows the work has actually been read and thought through, not just skimmed and started. It's a key difference between reacting to instruction and engaging with them
- **Aligns Expectations:** It forces a clear and concise conversation about scope, edge cases, and intent before those things become disputed later

- How To Do It In Practice

- Read through everything provided once fully before annotating, to understand the whole picture first
- Go through a second time and mark: unclear requirements, missing information, implicit assumptions, and open questions
- Write out the assumptions you're planning to make if something isn't answered before you start

- Send the markup back to the employer or stakeholder and get explicit confirmation or answers before beginning work
- Treat any project without a markup on file as incomplete preparation — not a shortcut, but a defect to correct before proceeding

- **Simple Examples**

Two small images of what a markup actually could look like in practice will be below; a UI mockup and a written ticket, each annotated before any work begins

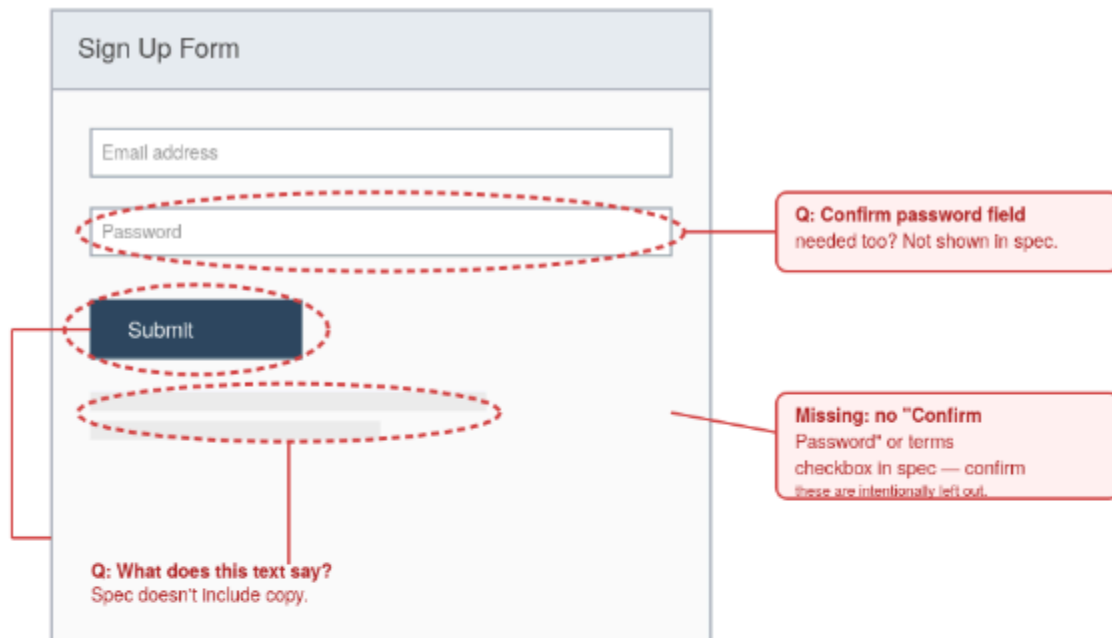


Figure 1 — Markup of a UI mockup: circled elements flag ambiguity, notes capture the open question.

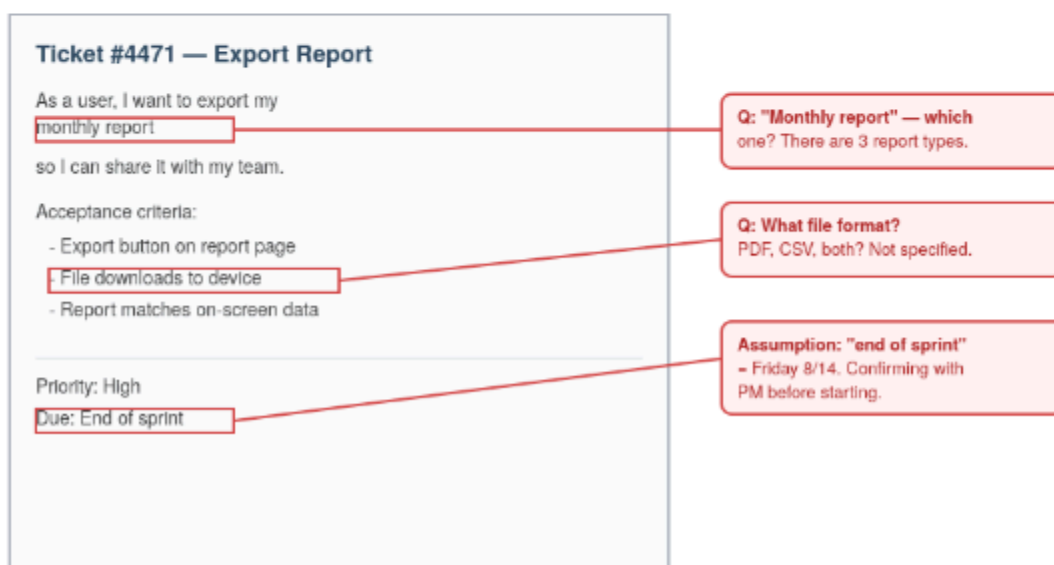


Figure 2 — Markup of a written ticket: boxed phrases show ambiguity, margin notes hold the questions and stated assumptions.